

Code Explanation

Delta Live Tables Test Suite

This test suite is designed to unit test the individual transformations that would be applied in a Delta Live Tables (DLT) pipeline. The tests cover the customer silver transformation, order silver transformation, and the join logic used in the ordersummary DLT table.

Overview

The test suite uses PySpark to create sample data, apply transformations, and verify the results. It tests the transformations that would be applied in the DLT pipeline, ensuring that the data is correctly filtered, deduplicated, and joined.

Step 1: Setting Up the Test Environment

The test environment is set up by creating a Spark session and a temporary directory for test data.

```
@classmethod
def setUpClass(cls):
    cls.spark = (SparkSession.builder
                  .appName("TestDeltaLiveTables")
                  .master("local[*]")
                  .config("spark.sql.extensions", "io.delta.sql.Delta
SparkSessionExtension")
                  .config("spark.sql.catalog.spark_catalog", "org.apa
che.spark.sql.delta.catalog.DeltaCatalog")
                  .getOrCreate())
    cls.temp_dir = tempfile.mkdtemp()
    cls.create_sample_data()
```

Step 2: Creating Sample Data

Sample customer and order data are created using PySpark DataFrames.

```
@classmethod
def create_sample_data(cls):
    customer_data = [
        (1, "John Doe", "john@example.com", "North"),
        (2, "Jane Smith", "jane@example.com", "South"),
        # ...
    ]
    customer_schema = StructType([
        StructField("CustId", IntegerType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])
    cls.customer_df = cls.spark.createDataFrame(customer_data, schem
a=customer_schema)

    order_data = [
        (101, "Laptop", 1200.0, 1, "2023-01-15", 1),
```

```

        (102, "Phone", 800.0, 2, "2023-01-20", 2),
        # ...
    ]
    order_schema = StructType([
        StructField("OrderId", IntegerType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", StringType(), True),
        StructField("CustId", IntegerType(), True)
    ])
    cls.order_df = cls.spark.createDataFrame(order_data, schema=order_schema)

```

Step 3: Testing Customer Silver Transformation

The customer silver transformation is tested by applying the transformation logic to the sample customer data and verifying the results.

```

def test_customer_silver_transformation(self):
    silver_customer_df = (
        self.customer_df
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
        .dropDuplicates(["CustId"])
    )
    self.assertEqual(silver_customer_df.count(), 4)
    self.assertEqual(silver_customer_df.filter(F.col("CustId") == 5).count(), 0)

```

Step 4: Testing Order Silver Transformation

The order silver transformation is tested by applying the transformation logic to the sample order data and verifying the results.

```

def test_order_silver_transformation(self):
    silver_order_df = (
        self.order_df
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
        .dropDuplicates(["OrderId"])
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
    )
    self.assertEqual(silver_order_df.count(), 6)

```

```

        self.assertEqual(silver_order_df.filter(F.col("OrderId") == 107)
            .count(), 0)

        laptop_order = silver_order_df.filter(F.col("ItemName") == "Laptop")
            .collect()[0]
        self.assertEqual(laptop_order["TotalAmount"], 1200.0)

        keyboard_order = silver_order_df.filter(F.col("ItemName") == "Keyboard")
            .collect()[0]
        self.assertEqual(keyboard_order["TotalAmount"], 150.0)

```

Step 5: Testing Order Summary Join

The order summary join is tested by applying the join logic to the silver customer and order DataFrames and verifying the results.

```

def test_ordersummary_join(self):
    silver_customer_df = (
        self.customer_df
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
        .dropDuplicates(["CustId"])
    )

    silver_order_df = (
        self.order_df
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
        .dropDuplicates(["OrderId"])
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
    )
    # Join logic to be implemented

```